Hands-On Practical Exam

DevSecOps — Secret Scanning

Build a GitHub Actions workflow that detects & blocks leaked secrets

Level: Junior DevSecOps · Infrastructure & Security Team

Exam Code	SEC-SS-GHA-01 (Junior)
Format	Hands-on coding — write a real GitHub Actions workflow
Time Allowed	60 minutes
Deliverable	Push your workflow to the provided GitHub repository
Tools Allowed	GitHub Docs, Gitleaks Docs, internet search — all allowed

■ Before you start

- Your starter repository is at: **github.com/flowaccount/DevSecOps-recruitment-challenge**
- Fork the repository to your own GitHub account, then clone it locally.
- The repo contains a Node.js app that has **secrets hardcoded in the source files** — intentionally.
- Your job is to create a GitHub Actions workflow file inside that repository.
- **Do not fix or remove the hardcoded secrets** — your workflow must detect them as-is.
- **You are free to use any secret scanning tool** you are comfortable with.
- Test your workflow by opening a Pull Request from a feature branch to `main`.
- Read each task carefully — they build on each other.

The Problem You Are Solving

The situation:

FlowAccount's `billing-service` repository currently has **no protection against secrets being pushed to GitHub**. A developer can accidentally commit an AWS key or database password, and nobody will notice until it is too late.

You have been asked to implement a GitHub Actions workflow that acts as an automated security gate on every Pull Request to `main`. If a secret is found, the PR must be blocked and the developer must be told exactly where the problem is and how to fix it.

■ Secrets hidden in the starter repo (find all of them):

File	Type of Secret
`src/config.js`	`AWS Access Key ID + Secret Access Key`
`src/config.js`	`Stripe live secret key + webhook secret`
`src/db.js`	`PostgreSQL database password`

Your 4 Tasks

Create a single file at: `.github/workflows/secret-scan.yml`
All 4 tasks must be implemented inside this one workflow file.

■ Task 1 — Scan for secrets on every Pull Request

Write a GitHub Actions workflow that triggers automatically whenever a Pull Request is opened, updated, or reopened targeting the `main` branch. The workflow must scan the repository for hardcoded secrets. **You are free to choose any secret scanning tool you prefer** — there is no required tool for this task.

■ Deliverable: A workflow file at .github/workflows/secret-scan.yml that triggers on PR events to main and scans for secrets using your chosen tool.

■ Task 2 — Block the merge if secrets are found

If your scanning tool finds any secrets, the workflow must **fail**. A failing workflow causes GitHub to block the Pull Request from being merged into main. Test this by opening a Pull Request — the merge button should show "Some checks were not successful" when secrets are present.

■ **Deliverable:** The workflow exits with a failure status when secrets are detected, and the PR merge is visibly blocked in GitHub.

■ Task 3 — Post a PR comment with findings and fix instructions

When secrets are found, the workflow must automatically post a **comment on the Pull Request** that includes: • Which files contain secrets • What type of secret was found in each file • A clear step-by-step instruction on how to fix it The comment should be helpful and actionable — the developer reading it should know exactly what to do next. **You decide how to implement the comment** — use any GitHub Actions approach you prefer.

■ **Deliverable:** An automated PR comment appears when secrets are detected, listing affected files and providing clear remediation steps.

■ Task 4 — Explain your workflow design

Write a step-by-step explanation of how your workflow operates. You can write this as a comment block inside the YAML file, or in a separate `ANSWER.md` file at the repo root. Cover the following points: 1. Why did you choose your scanning tool? 2. What does each step in your workflow do? 3. How does the workflow block the merge? 4. How does it post the PR comment? 5. What would you improve or add if you had more time?

■ **Deliverable:** A written explanation inside the YAML as comments, or in ANSWER.md in the repo root.

Workflow Design — Your Approach

You have full freedom to design and implement this workflow.

There is no single correct answer. We evaluate your workflow on whether it works, how clearly it is structured, and how well you can explain your decisions.

■ What your workflow must achieve:

1	Triggers on every Pull Request targeting main
2	Scans the repository for hardcoded secrets
3	Fails the workflow (and blocks merge) when secrets are found
4	Posts a PR comment listing affected files and how to fix them

■ Things to think about as you build:

Tool choice	Which secret scanning tool will you use, and why? Consider ease of integration with GitHub Actions, output format, and community support.
Failure behaviour	How does your workflow signal failure to GitHub? What exit code or condition makes the merge button turn red?
PR comment timing	The comment step must run even when a previous step fails. How do you control when a step runs in GitHub Actions?
Comment content	A developer will read your comment and needs to know exactly what to fix. How will you include the affected file names and clear remediation steps?
Permissions	Posting a comment on a PR requires specific GitHub Actions permissions. What permissions does your workflow need to declare?

How to Submit

1	Go to github.com/flowaccount/DevSecOps-recruitment-challenge
2	Click Fork and fork the repository to your own GitHub account
3	Clone your fork: git clone https://github.com/YOUR-USERNAME/DevSecOps-recruitment-challenge
4	Create a new branch: git checkout -b feature/secret-scanning
5	Create your workflow file at .github/workflows/secret-scan.yml
6	Add your explanation to ANSWER.md in the repo root (or as YAML comments)
7	Push your branch and open a Pull Request to main on your fork
8	Share the Pull Request URL with your interviewer